

Documentación Técnica: Sistema de Buffs y Estados Temporales

Este documento detalla la arquitectura del sistema de Buffs y Estados Alterados (Debuffs) en **Visceral Limbo**. Está diseñado como una guía estandarizada para que todo el equipo de desarrollo pueda implementar nuevos efectos de forma modular, escalable y respetando la arquitectura del proyecto basada en `ScriptableObject` y principios SOLID.

1. Introducción al Sistema

El sistema permite aplicar modificaciones temporales o pasivas a las entidades (jugador o enemigos). Está fuertemente tipado y dividido en tres piezas fundamentales para garantizar que no se introduzcan dependencias ocultas o *hardcoding*:

- **BuffBehavior** : Clase base abstracta. Contiene la lógica activa del buff (qué sucede cada frame, al aplicarse o al expirar).
- **BuffSO** : `ScriptableObject` que actúa como un contenedor de datos inmutable. Almacena la "metadata" (duración, identificador, comportamiento al acumular *stacks* y referencias de clase).
- **BuffManager** : Componente adosado a la entidad que administra los diccionarios de buffs activos, controla los temporizadores y gestiona la instanciación o destrucción segura de los comportamientos.

2. Paso a Paso: Añadir un Nuevo Buff

Para crear un nuevo efecto en el juego (ej. quemadura, envenenamiento, o escudos), el flujo de trabajo es el siguiente. **No es necesario modificar el código del `BuffManager`** para agregar nuevos efectos, respetando así el principio de Abierto/Cerrado (Open/Closed).

Paso 1: Programar el Comportamiento (Script)

Crea un nuevo script que herede de la clase abstracta `BuffBehavior`. El compilador exigirá la implementación de cuatro métodos clave:

```
public class MiNuevoBuff : BuffBehavior
{
    // 1. Se llama UNA VEZ cuando el buff se adhiere a la entidad
    public override void OnApply(BuffManager manager, StatsManager StatMan)
    {
        // Ideal para inicializar referencias. Ej:
```

```

        // _Hp = manager.HPComponent;
    }

    // 2. Se llama cuando se añade potencia (stacks) al buff existente
    public override void OnAddPotency(int ExtraPotency)
    {
        _BuffPotency += ExtraPotency;
        // Recalcular daño o curación en base a la nueva potencia
    }

    // 3. Lógica continua (se llama cada frame por el BuffManager)
    public override void OnUpdate(float deltaTime)
    {
        // Usar acumuladores de tiempo para efectos periódicos (Ticks)
    }

    // 4. Se llama al finalizar el tiempo del buff o si se fuerza su expiración
    public override void OnExpire()
    {
        // Crucial para limpiar efectos visuales o revertir estadísticas
        alteradas
    }
}

```

Paso 2: Configurar el ScriptableObject (Data)

Una vez que el script compila, ve al Editor de Unity y crea un nuevo **BuffSO** haciendo clic derecho en la ventana Project (*Create > Visceral_Limbo > General > BuffSystem > BuffScriptableObject*).

Configura los campos críticos:

- **Buff Type:** Selecciona si es un **Buff** o un **Debuff**. Esto determinará a qué diccionario interno irá.
- **Buff ID:** Define un string único (ej. **_VenenoFuerte**). Es la llave para gestionar instancias repetidas.
- **Buff Duration:** Tiempo en segundos. Si es un efecto permanente, marca la casilla *Is Infinity Duration*.
- **Should Override / Should Scale:** Configuración excluyente. Define si al aplicar el mismo buff dos veces este reemplaza al anterior reseteando el timer, o si le suma potencia al buff ya existente.
- **Buff Behavior Script: ¡CRÍTICO!** Arrastra tu nuevo script a este campo. Gracias a la lógica en el **OnValidate**, el SO guardará automáticamente el **AssemblyQualifiedName**. Esto permite instanciar el componente dinámicamente sin usar strings quemados en código.

Paso 3: Aplicación a través del Manager

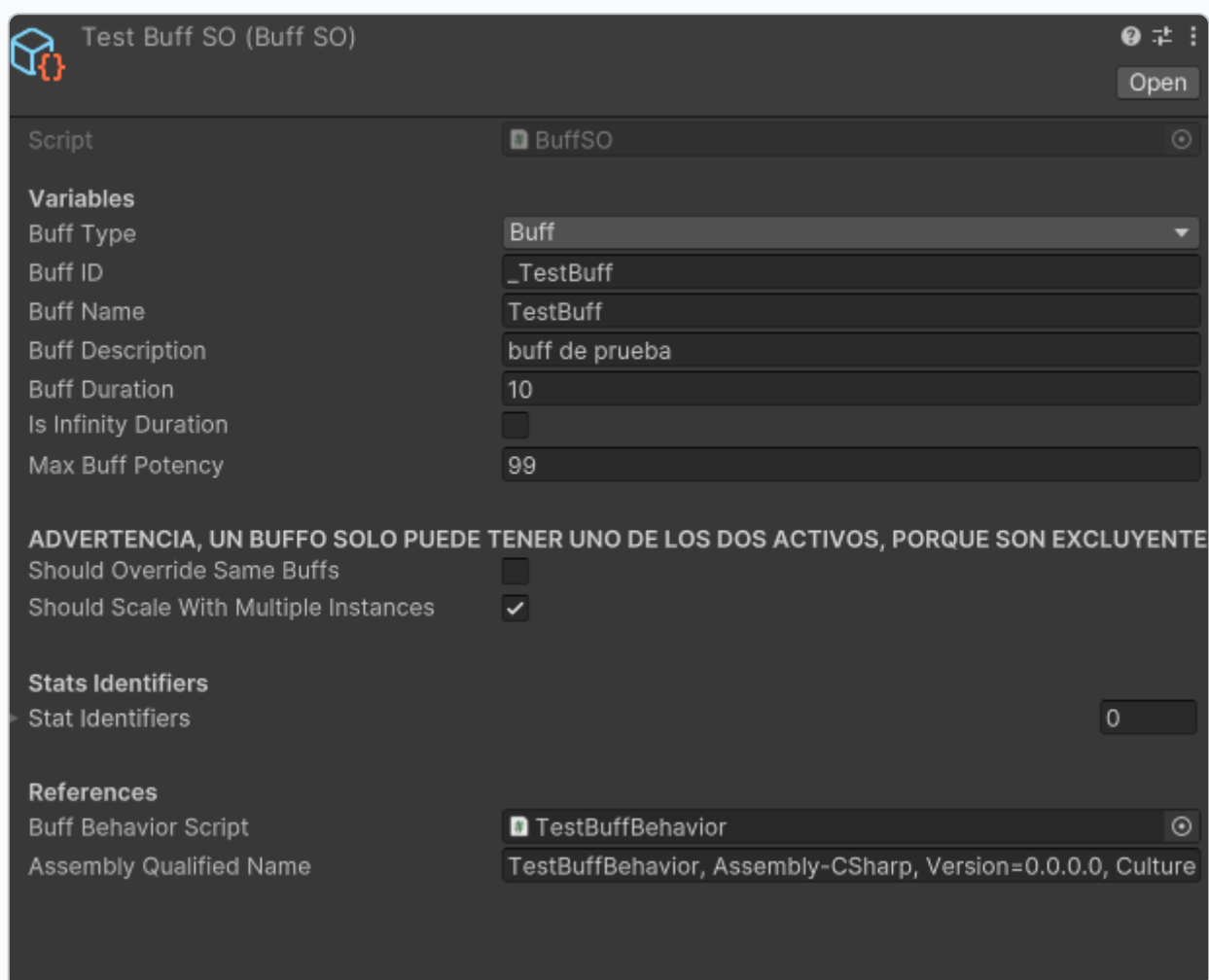
Desde el sistema de combate (un arma, un hitbox enemigo, etc.), accede al **BuffManager** del objetivo y ejecuta:

```
// ID del Buff, Objeto de Datos (BuffSO), Potencia a aplicar, Entidad Causante  
targetBuffManager.AddNewBuff(miBuffSO.BuffID, miBuffSO, 1, inflictorContext);
```

3. Caso Práctico (TestBuffBehavior)

Analizando la implementación real provista en el proyecto, veamos cómo el **TestBuffBehavior** se integra en el sistema. Este buff de prueba aplica una curación periódica y, de forma engañosa, inflige daño de golpe al expirar.

Configuración en el Inspector



Como se observa en el asset `Test Buff S0` , se ha asignado **Buff Type: Buff**, con una duración de 10 segundos. Se marcó **Should Scale With Multiple Instances** para que re-aplicar el buff sume potencia curativa en lugar de reiniciarlo. Finalmente, el campo *Buff Behavior Script* apunta a `TestBuffBehavior` , generando su firma de ensamblado.

Implementación Lógica

En el código (`TestBuffBehavior.cs`), se observa un patrón excelente para optimización: en lugar de procesar curación cada frame, se usa un acumulador de tiempo (`pulse`) contra un `timer` :

```
public override void OnUpdate(float deltaTime)
{
    if(pulse < timer)
    {
        pulse += deltaTime;
        return; // Retorna rápido, ahorrando procesamiento
    }
    else
    {
        pulse = 0;
        _HP.HealHP(HPHealth); // Aplica el efecto de curación
    }
}
```

4. Guía Rápida de Uso e Implementación

Flujo de Trabajo Resumido

1. **Determina el Efecto:** ¿Es pasivo (modifica un Stat), activo continuo (Ticks de curación/daño) o un evento único retrasado?
2. **Crea la Clase:** Hereda de `BuffBehavior` e implementa los 4 métodos abstractos.
3. **Inicialización Limpia:** Utiliza `OnApply` para buscar dependencias locales en la entidad (ej. `Health_Component`). No uses `Awake()` o `Start()`.
4. **Manejo de Tiempos:** En `OnUpdate`, suma el `deltaTime` para crear sub-temporizadores lógicos (procs).
5. **Crea los Datos:** Genera la instancia de `BuffSO` en las carpetas de recursos.
6. **Asocia el Código:** Arrastra el script recién creado al campo de *References* en el Inspector para que enlace.
7. **Despliega:** Llama a `AddNewBuff()` enviando el SO, la potencia y el contexto del atacante desde tu controlador de armas o daños.

5. Buenas Prácticas de Arquitectura

- **Evita los Métodos de Unity Base:** Al instanciar componentes dinámicamente, el orden de `Start()` y `Awake()` puede ser impredecible frente a la inyección de datos. Utiliza exclusivamente `OnApply()` para configurar tu buff, ya que te asegura que las referencias (`BuffManager` y `StatsManager`) ya están inyectadas.
- **Soporte para Dilation Time:** Si estás creando debuffs de daño continuo (como en `BleedDebuff.cs`), asegúrate de multiplicar el tiempo por el factor global: `deltaTime * TimeDilationManager.GlobalTimeScale`. Esto garantiza que los efectos de estado respeten mecánicas como *bullet time*, aturdimiento de entidades o pausas del sistema de juego.
- **Manejo Responsable de Memoria y Estados:** Todo cambio directo a una estadística, material visual o escala de mallas que realices en `OnApply` **debe ser revertido obligatoriamente en `OnExpire()`**. Fallar en este principio dejará entidades en estados corruptos de por vida y generará Memory Leaks si hay suscripciones a delegados sin remover.
- **Trazabilidad del Daño (PlayerContext):** El manager soporta un objeto `Origin`. Cuando inflijas daño a través de un buff (ej. sangrado o la explosión del `TestBuffBehavior` al expirar), asegúrate de poblar correctamente los struct de daño (ej. `DamageScore`) asignando el `Attacker = Origin` y el `FactionID`. Esto permite que los sistemas de *on-kill effects* (ej. robar vida al matar a un enemigo por sangrado) sigan funcionando de manera predecible.